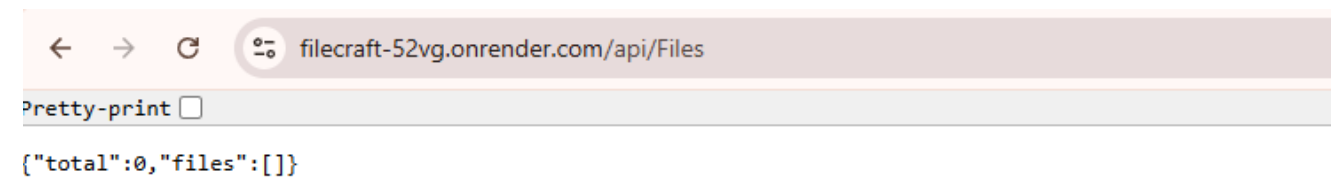


Публічне посилання: <https://filecraft-52vg.onrender.com>

Скріншот відповіді /health endpoint у браузері:



Скріншот успішної відповіді основного API-endpoint



Скріншот dashboard хмарної платформи:

The screenshot displays the FileCraft dashboard on the Render.com platform. The interface includes a sidebar with navigation options like Environment, FileCraft, Events, Settings, MONITOR (Logs, Metrics), and MANAGE (Environment, Shell, Scaling, Previews, Disk, One-Off Jobs). The main content area shows the 'FileCraft' service, which is a 'WEB SERVICE' using 'Docker' and is currently 'Free'. It provides the Service ID, source code link, and a URL to the live application. A notification banner states: 'Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.' Below this, a log entry from June 11, 2026, at 12:37 AM shows the service is 'Live'. A 'Logs' section displays a series of log messages from the application, including startup information and a warning about a failed redirect. A modal window on the right, titled 'FileCraft is live!', encourages upgrading to a Pro workspace for additional features like log retention and autoscaling.

конфігураційні файли (скріншоти, всі файл завантажені на GitHub):

Схема: <https://www.schemastore.org/appsettings.json>

```
1  {
2    "Logging": {
3      "LogLevel": {
4        "Default": "Information",
5        "Microsoft.AspNetCore": "Warning"
6      }
7    },
8    "AllowedHosts": "*"
9  }
10
```

Cxema: <https://json.schemastore.org/launchsettings.json>

```
1  {
2    "$schema": "https://json.schemastore.org/launchsettings.json",
3    "profiles": {
4      "http": {
5        "commandName": "Project",
6        "dotnetRunMessages": true,
7        "launchUrl": "swagger",
8        "launchBrowser": true,
9        "applicationUrl": "http://localhost:5252",
10       "environmentVariables": {
11         "ASPNETCORE_ENVIRONMENT": "Development"
12       }
13     },
14     "https": {
15       "commandName": "Project",
16       "dotnetRunMessages": true,
17       "launchUrl": "swagger",
18       "launchBrowser": true,
19       "applicationUrl": "https://localhost:7057;http://localhost:5252",
20       "environmentVariables": {
21         "ASPNETCORE_ENVIRONMENT": "Development"
22       }
23     }
24   }
25 }
26
```

<Project Sdk="Microsoft.NET.Sdk.Web">

```
<PropertyGroup>
  <TargetFramework>net10.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ImplicitUsings>enable</ImplicitUsings>
</PropertyGroup>
```

```
<ItemGroup>
  <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="10.0.8" />
  <PackageReference Include="Swashbuckle.AspNetCore.SwaggerUI" Version="10.2.1" />
</ItemGroup>
```

```
<ItemGroup>
  <ProjectReference Include="..\FileSharingSystem\FileSharingSystem.csproj" />
</ItemGroup>
```

</Project>

```
Dockerfile
1 # Stage 1: Build Environment
2 FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
3 WORKDIR /src
4
5 # Copy the project files maintaining their folder structure
6 COPY ["FileCraftAPI/FileCraftAPI.csproj", "FileCraftAPI/"]
7 COPY ["FileSharingSystem/FileSharingSystem.csproj", "FileSharingSystem/"]
8
9 # Restore dependencies for the main API (this automatically restores referenced projects too)
10 RUN dotnet restore "FileCraftAPI/FileCraftAPI.csproj"
11
12 # Copy the rest of the source code for all projects
13 COPY . .
14
15 # Change working directory to the API project to build it
16 WORKDIR "/src/FileCraftAPI"
17 RUN dotnet build "FileCraftAPI.csproj" -c Release -o /app/build
18
19 # Stage 2: Publish
20 FROM build AS publish
21 RUN dotnet publish "FileCraftAPI.csproj" -c Release -o /app/publish /p:UseAppHost=false
22
23 # Stage 3: Final Runtime Environment
24 FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
25 WORKDIR /app
26 EXPOSE 8080
27
28 COPY --from=publish /app/publish .
29
30 # Start the API
31 ENTRYPOINT ["dotnet", "FileCraftAPI.dll"]
```

Посилання на Git-репозиторій з кодом (включно з модулем із ЛР 02–04, тестами): <https://github.com/yuliapanasenko-source/FileCraft>

Бонус: файл .github/workflows/deploy.yml:

```
1 name: .NET CI/CD
2
3 on:
4   push:
5     branches: [ "main" ]
6   pull_request:
7     branches: [ "main" ]
8
9 jobs:
10   build-test-deploy:
11     runs-on: ubuntu-latest
12
13     steps:
14       - uses: actions/checkout@v4
15
16       # Updated to .NET 10
17       - name: Setup .NET
18         uses: actions/setup-dotnet@v4
19         with:
20           dotnet-version: '10.0.x'
21
22       # Running these commands at the root level will automatically find
23       # your .sln file and build both FileCraftAPI and FileSharingSystem
24       - name: Restore dependencies
25         run: dotnet restore
26
27       - name: Build
28         run: dotnet build --no-restore -c Release
29
30       - name: Test
31         run: dotnet test --no-build -c Release --verbosity normal
32
33       # The deployment step only runs if the previous steps succeed,
34       # and ONLY if this action was triggered by a direct push/merge to main.
35       # It will safely ignore pull requests.
36       - name: Trigger Render Deployment
37         if: success() && github.event_name == 'push' && github.ref == 'refs/heads/main'
38         run: curl -X POST ${ secrets.RENDER_FC }
```